

Reinforcement Learning for Walking Loop Generation

Francesco D'Aprile

08/06/2026

1 Introduction

The goal of this project is to train an agent that builds circular walking routes on a real city map. A route starts and ends at a fixed home node and should last close to a target duration given in minutes. The agent works on the pedestrian graph of a city, picks points of interest (POIs) one at a time, and decides when to go back home. I trained it with PPO and action masking. This report explains the choices I made in the current version, the things I tried that did not work, and what could be done next.

Note: I delivered the project as a single big notebook because I red wrongly, so I collapsed the last version of the entire project in this notebook, I hope that the pipeline, the results and the visualizations are clear. Unfortunately I had to compress the notebook pdf for the delivery, I hope there will not problems for it

2 Implementation choices

2.1 Graph and POIs

I downloaded the pedestrian graph with 3km radius of each city from OpenStreetMap with OSMnx and kept only the largest strongly connected part. This way every node can reach every other node, so it never gets stuck with an impossible path.

Letting the agent choose any node of the graph would give a huge action space, which PPO cannot handle well. Instead I sample a fixed set of 80 candidate POIs per city and the agent only chooses among them. This keeps the problem small enough to learn.

I also precompute the shortest paths only between the key nodes, that is the home node and the 80 POIs, using one single source Dijkstra from each of them, and I cache the result.

2.2 Environment

The observation is a single vector. It contains a few scalar values: the target duration, the current duration of the loop so far, the time needed to go home right now, the remaining time budget, and an urgency value that grows when there is little time left. It also contains the position of home relative to the current node, and the last three visited nodes in relative coordinates (breadcrumbs), which give the agent a sense of where it is going. Finally, for each POI it contains whether the POI was already visited, its relative position, the time to reach it, and a feasibility value that says whether the POI can still be visited while leaving enough time to return home.

The action is discrete. The agent either picks one of the POIs to visit next, or chooses to go home, which ends the episode and closes the loop.

I train on several cities at the same time so the agent does not overfit to a single map. At the start of each episode I pick the city with a probability proportional to its recent error, kept as an exponential moving average. In this way the cities that the agent finds harder are seen more often during training.

At every reset I also shuffle the order of the POIs. Because of this, the action index has no fixed meaning, so the agent is forced to look at the POI features to decide, instead of memorising positions. This fits the attention policy very well, since attention does not depend on the order of the inputs, so the shuffle does not confuse it and it still learns a stable behaviour.

For the action masking I only remove two kinds of actions: POIs that were already visited, and the POI that is at the current position. These actions make no sense, so masking them does not change the task, it just stops the agent from wasting a step.

2.3 Reward

The reward is a sum of a few terms. Some are given at every step, others only when the loop ends.

While the agent walks from one POI to the next, it gets an exploration term along the streets it crosses: +0.3 for each new node, -0.3 for each node it had already visited, and -0.1 for each edge it walks a second time. This sum is clipped from below at -3, so one bad step cannot dominate the reward.

At every step that is not the final one, it also gets a dense timing term. If the agent went home right now, the loop would last $t_{proj} = t_{cur} + t_{home}$, where t_{home} is the time to return home from the current node. With the projected relative error

$$e_{proj} = \frac{|t_{target} - t_{proj}|}{t_{target}},$$

this term is $0.1 \cdot \max(0, 1 - 2e_{proj})$. It is largest when going home now would land on the target, and it falls to zero once the projected error passes 50%. A penalty of -0.5 is added if the agent is so far that it could no longer get home in time.

When the agent chooses to go home, the loop ends and it gets the two terminal terms. The main one is the time precision. With the final relative error $e = |t_{target} - t_{actual}|/t_{target}$,

$$r_{time} = \max(-30, 100 - 200e).$$

A perfect loop gives 100, the value drops as the error grows, and it is clipped at -30.

The second terminal term rewards the shape of the loop. Let $c \in [0, 1]$ be the circularity score, computed from the loop area with the Shoelace formula. The bonus is

$$r_{circ} = c \cdot 25 \cdot \max\left(0, 1 - \frac{e}{0.5}\right).$$

The last factor ties the shape to the timing: it is full when the error is zero and fades to zero at 50% error, so the agent only earns the shape bonus when the duration is also good since it is the priority.

Finally, if the episode is cut off without going home, because it used too many steps or went well over the target, it gets a penalty of -5.

2.4 Policy

The policy is an attention based pointer network. Each POI is encoded independently by a small shared network, and the current state is encoded into a context vector. The context then attends

over the POI embeddings, and the score of each POI, which is its action logit, comes from how well the context matches that POI. A separate head produces the logit for the home action. The same encoder feeds both the actor and the critic. I chose this design because the POIs are an unordered set, and attention handles sets naturally.

2.5 Training

I train with MaskablePPO from sb3-contrib, which supports action masking directly, and I run 16 environments in parallel with SubprocVecEnv to collect data faster, since the graph simulation runs on CPU. I also use a three phase curriculum on the target duration. Training starts with long targets in $[60, 120]$ minutes, which are easier because there is space for many POIs, then opens to $[30, 120]$, and finally to the full range $[10, 120]$. Short targets are the hardest, so I leave them for last.

I ran and trained everything on a remote machine with an NVIDIA RTX 3090 and 24 GB of VRAM. With the current parameters one full training run takes about half an hour. In the figure 1 and 2 there are represented respectively the learning curve with the reward graph and the loss curve during training. It is clear that the training is a little instable but I didn't manage to get better graphs without degrading the performances of the agent. But they show that both agent and attention model understood the task having positive graphs.

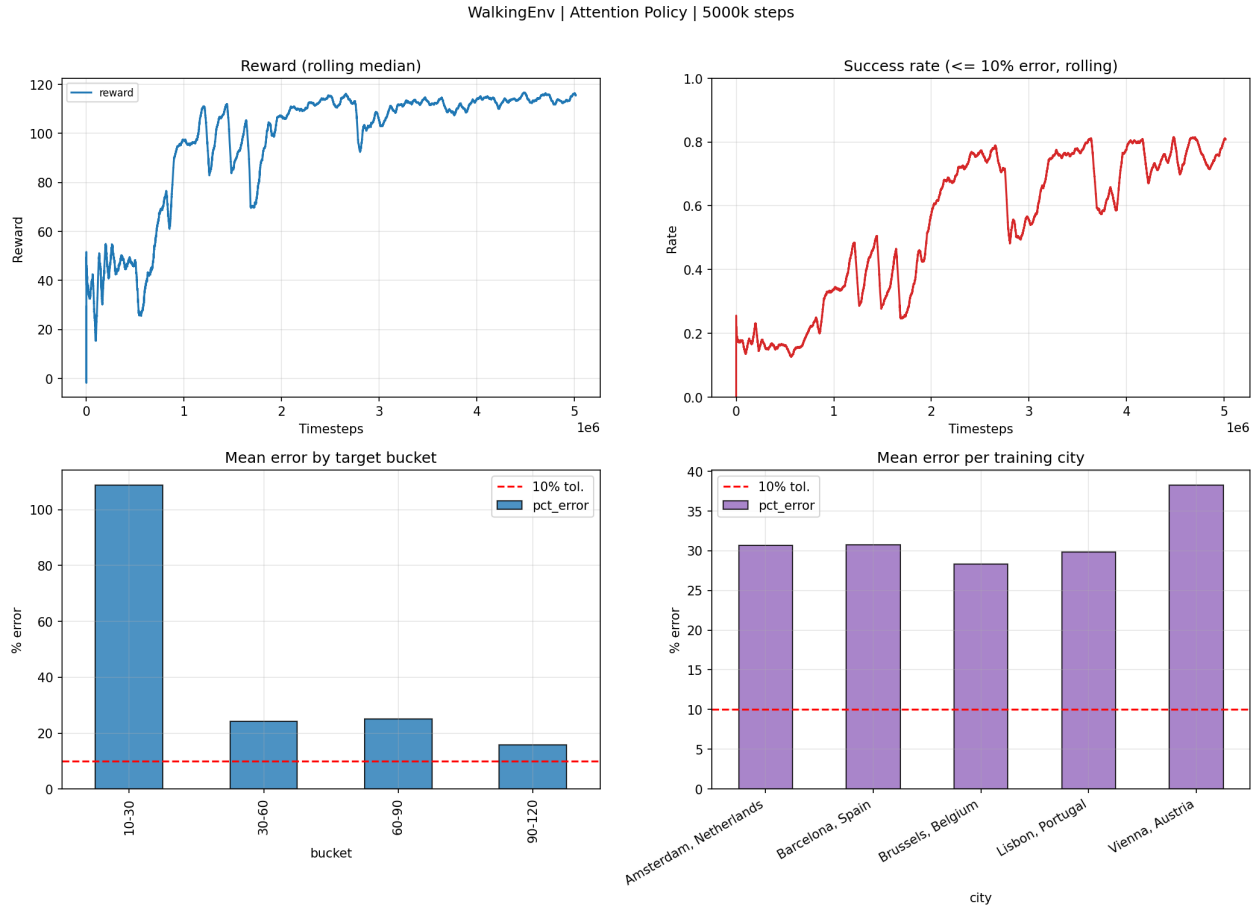


Figure 1: Learning curve during training

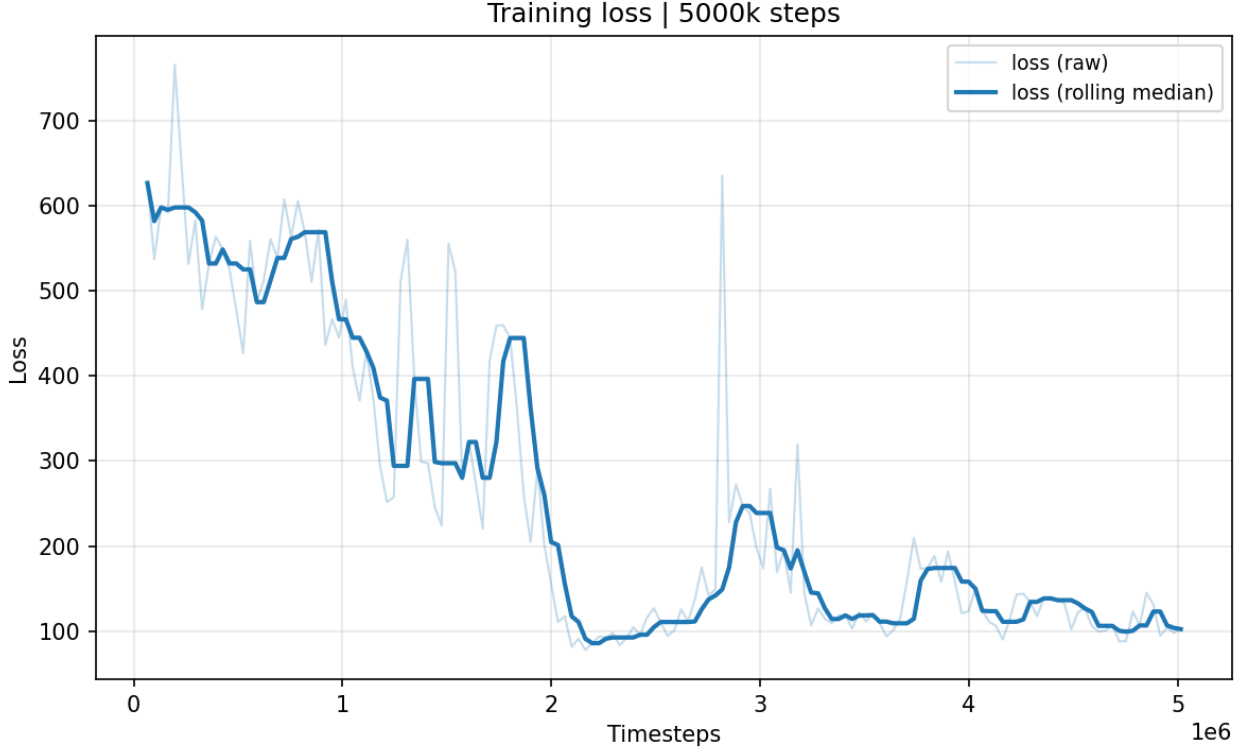


Figure 2: Loss curve during training

3 Evaluation

I evaluate the agent only on cities that were never seen during training, so the numbers measure how well it generalises and not how well it memorised. I count a loop as a success when its error is below 10%. This 10% threshold was chosen empirically by me and is not a standard value.

On a test set of 100 loops the agent reaches a mean error of 7.24% and a median error of 4.09%, and stays within 10% of the target on 83% of the loops. The mean circularity is 0.102 and the mean edge revisit ratio is 0.265. Table 1 splits these results by city, and Table 2 splits them by target duration.

City	Mean error (%)	Success $\leq$ 10%	n
Bruges, Belgium	4.69	0.95	20
Copenhagen, Denmark	14.08	0.55	20
Florence, Italy	5.89	0.90	20
Prague, Czech Republic	4.20	1.00	20
Rome, Italy	7.36	0.75	20

Table 1: Evaluation results per city, all on cities not seen during training.

The agent generalises well to cities it never saw during training. Copenhagen is the hardest city, while on Prague almost every loop is a success. The short loops, in the 10 to 30 minute range, are clearly the worst, but the error stays acceptable. These short targets are hard to match because even a couple of POIs already use up most of the time budget. For the longer ranges the error drops to just a few percent.

City	Target (min)	Mean error (%)	Success $\leq 10\%$	Circularity	n
Bruges, Belgium	10-30	12.39	0.80	0.027	5
Bruges, Belgium	30-60	2.20	1.00	0.075	6
Bruges, Belgium	60-90	1.24	1.00	0.150	2
Bruges, Belgium	90-120	2.31	1.00	0.076	7
Copenhagen, Denmark	10-30	32.12	0.25	0.000	4
Copenhagen, Denmark	30-60	17.61	0.40	0.046	5
Copenhagen, Denmark	60-90	10.34	0.40	0.070	5
Copenhagen, Denmark	90-120	2.23	1.00	0.116	6
Florence, Italy	10-30	12.16	0.75	0.069	4
Florence, Italy	30-60	6.58	0.86	0.100	7
Florence, Italy	60-90	2.15	1.00	0.103	4

Table 2: Evaluation results split by target duration range.

Beyond the evaluation conducted across different cities and time intervals, the end of the notebook includes a script that allows the user to specify a particular city and a desired walking duration. The program then computes three routes and displays them graphically (Figure 1), together with the target time, the actual route duration, the percentage error, and the circularity factor.

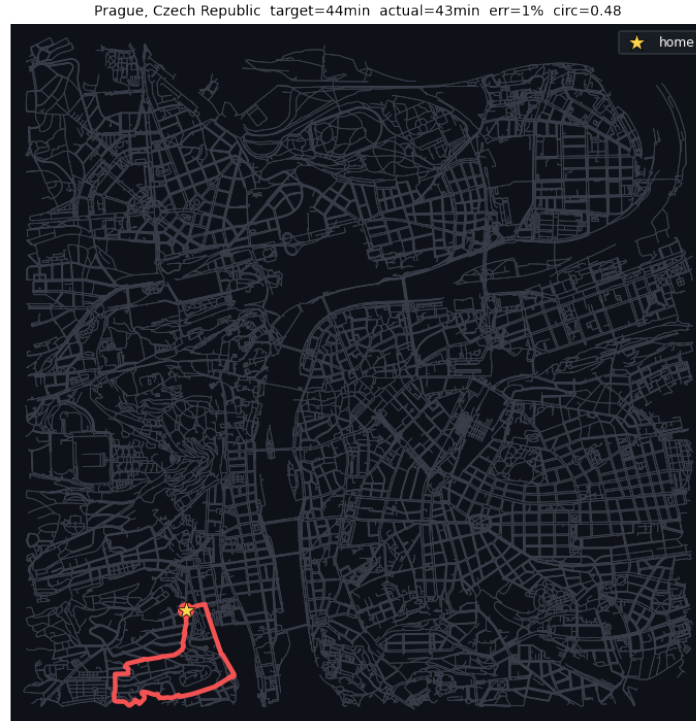


Figure 3: Example of the route visualization

4 Human feedback fine-tuning

On top of the trained model I tried to add a way to adapt the routes to personal taste. The user looks at a few generated loops and gives each one a score from 1 to 10, based on how much he likes it. From these scores I fit a small ridge regression model that predicts the score from seven geometric statistics of the loop: circularity, node revisit ratio, mean turning angle, compactness, edge revisit ratio, percentage error, and number of POIs. This model is a critic that learns, in an implicit way, how much the user cares about each of these aspects from the votes alone. I then keep training the agent for a short time, adding the predicted score (scaled by a factor) to the final reward, so the policy slowly moves towards the kind of loops the user prefers.

In my tests the fine-tuned model produced loops with a better shape. I also noticed that if I run too many fine-tuning rounds the agent can start to forget the time objective, a form of catastrophic forgetting. I did not study this effect in depth, it is just something I observed.

5 What did not work

For choosing the POIs I first tried k-means clustering on the node coordinates. It is efficient and cleaner in theory, but it spreads the points uniformly over the map. What I actually wanted was more points near home and fewer far away, so I dropped it and used a distance based selection instead.

My first policy was a simple MLP that took all the POI features concatenated into one flat vector. Since the POIs are shuffled every episode, the network had to relearn their meaning from a different position each time, so it learned slowly and was unstable. Moving to the attention policy fixed this.

I also tried a graph neural network as the encoder. It gave decent results, around 74% on new cities, but it needs the whole graph as input, extra preprocessing and a lot of memory. The attention policy captures what really matters, the relations between the POIs, with much less complexity, so I kept that one.

At one point I removed two observation features, the remaining time budget and the urgency value, because they can be derived from the others. Learning got clearly worse, so I put them back. The redundancy turned out to be useful and made learning faster.

I tried a linear decay of the learning rate: the learning curve looked smoother, but the final results were much worse, so I went back to a constant learning rate.

I ran a hyperparameter search with Optuna. Starting from its suggestions, I then changed some values by hand, looking not only at the loss but also at the reward and at the quality of the routes, which is what I really care about.

6 Future work

A first direction is to train on more cities and to add domain randomisation, for example random radius and number of POIs. This should make the agent more robust on maps it has never seen.

Right now the home node is fixed for each city. Sampling a different home at each episode would let the agent start from any point, which is closer to real use but it should recompute the POIs each time.

The human feedback part could grow into a full personalisation system. Instead of one fixed model, the user would be described by a continuous preference vector, and the policy would be conditioned on it. A simple version of this idea is to build a user profile from a few feature

importances, for example how wide the route should be, how far from home, or how circular, and then use these values as multipliers for those terms in the reward.

Another direction is to enrich the POIs with semantic tags from OpenStreetMap, such as parks, cafes or green areas, so the routes can also take into account the type of place and not only the geometry.